

Informe de Aplicación de la Guía de Práctica Experimental

Aplicaciones Distribuidas — Entrega 4 (PFC)

GA-SUM-06 / PE-U5: CI/CD, Pruebas y Observabilidad

Harold Nicolás Vinueza Sánchez
Carrera de Software, Facultad de Ciencias de la Computación
Universidad Técnica Estatal de Quevedo (UTEQ)

25 de agosto de 2026

Resumen

(resumen de una página máx.: qué se hizo, sobre qué sistema, con qué resultado)

Índice

1. Introducción	2
2. Arquitectura en capas y patrones de diseño distribuido	2
2.1. Arquitectura hexagonal en <code>academico-laboratorios-service</code>	2
2.2. Patrón Repository	2
2.3. Patrón Facade	2
2.4. Patrón Singleton	3
3. Pruebas en sistemas distribuidos	3
3.1. Pirámide de pruebas del sistema	3
3.2. Pruebas sobre <code>academico-laboratorios-service</code>	3
3.3. Pruebas instrumentadas de la funcionalidad móvil de escaneo QR	3
3.4. Cobertura verificada en el resto de microservicios	3
4. DevOps: CI/CD con GitHub Actions	4
4.1. Estructura general del pipeline	4
4.2. Jobs de análisis estático y pruebas	4
4.3. Jobs de construcción y empaquetado	4
4.4. Job de integración completa	4
4.5. Limitación identificada	4
5. Observabilidad distribuida	5
5.1. Instrumentación implementada	5
5.2. Limitación identificada: campaña de carga no ejecutada	5
6. Evaluación de calidad ISO/IEC 25010:2023	5
7. Reflexión ética	5
8. Conclusiones	5

1. Introducción

El presente informe documenta la aplicación de la guía de práctica experimental correspondiente a GA-SUM-06 / PE-U5 — CI/CD, Pruebas y Observabilidad, de la asignatura Aplicaciones Distribuidas, sobre el Sistema de Control de Laboratorios Informáticos (SCLI), desarrollado como Proyecto de Fin de Curso por el equipo FUVV. El sistema sigue una arquitectura de microservicios desplegados de forma independiente y comunicados a través de un *API Gateway* [1], con persistencia distribuida sobre un clúster de CockroachDB.

Este documento se centra en el trabajo realizado por el autor sobre el microservicio `academico-laboratorios-service` responsable de la gestión del dominio académico del sistema (campus, facultades, carreras, laboratorios, equipos y horarios), así como en la aplicación transversal de las prácticas de integración continua, pruebas automatizadas y observabilidad distribuida al conjunto del proyecto, en la medida en que dichas prácticas fueron verificadas directamente sobre el repositorio del PFC.

El objetivo de este informe es doble: por un lado, evidenciar de manera honesta y verificable el estado real de implementación de cada práctica exigida por la guía; por otro, reflexionar críticamente sobre las decisiones técnicas tomadas, sus fundamentos teóricos y sus limitaciones, siguiendo el criterio de que toda medición reportada debe estar respaldada por evidencia reproducible y no por estimaciones.

2. Arquitectura en capas y patrones de diseño distribuido

2.1. Arquitectura hexagonal en `academico-laboratorios-service`

El microservicio `academico-laboratorios-service` fue refactorizado siguiendo el patrón de arquitectura hexagonal (*Ports and Adapters*), organizando el código en cuatro capas: dominio, aplicación, infraestructura y presentación. Las once entidades del dominio académico (Campus, Facultad, Carrera, Piso, Laboratorio, TipoEquipo, Equipo, Materia, PeriodoLectivo, Bloque y HorarioAcademico) se implementaron bajo este esquema, desacoplando la lógica de negocio de los detalles de persistencia y de exposición HTTP.

Esta separación responde a los principios SOLID aplicados a microservicios: el principio de responsabilidad única se refleja en la separación entre puertos de dominio y adaptadores de infraestructura, y el principio de inversión de dependencias se materializa mediante interfaces de repositorio definidas en la capa de dominio e implementadas en la capa de infraestructura, evitando que la lógica de negocio dependa de un motor de persistencia concreto.

2.2. Patrón Repository

Cada entidad del dominio académico cuenta con una interfaz de repositorio definida en la capa de dominio, con su implementación correspondiente en la capa de infraestructura sobre Spring Data JPA. Esta decisión permite sustituir el mecanismo de persistencia sin alterar la lógica de negocio, y facilita las pruebas unitarias mediante dobles de prueba de los puertos de repositorio, siguiendo el catálogo clásico de patrones de diseño reutilizables [2] adaptado a un contexto de microservicios [3].

2.3. Patrón Facade

El endpoint `GET /api/v1/laboratorios/{id}/detalle-completo` expone la vista consolidada de un laboratorio, que requiere coordinar información de cinco servicios de dominio distintos (laboratorio, piso, facultad, equipos y horarios). En lugar de exponer esta orquestación al cliente, se implementó el patrón Facade mediante la interfaz `LaboratorioDetalleFacade` y su implementación, que centraliza la coordinación de dichos servicios detrás de un único punto de entrada, reduciendo el acoplamiento entre la capa de presentación y la lógica de orquestación interna.

2.4. Patrón Singleton

El registro de métricas HTTP del servicio, `HttpRequestsMetricsRegistry`, se implementa como Singleton para garantizar una única instancia compartida de los contadores de Micrometer a lo largo del ciclo de vida de la aplicación, evitando la duplicación de series temporales en Prometheus.

3. Pruebas en sistemas distribuidos

3.1. Pirámide de pruebas del sistema

La estrategia de pruebas del PFC cubre varios niveles complementarios. En el nivel unitario, cada microservicio backend emplea JUnit 5 y Mockito, aislando la lógica de negocio de sus dependencias mediante dobles de prueba de los puertos de dominio definidos por la arquitectura hexagonal, sin requerir infraestructura real. En el nivel de integración, las pruebas se ejecutan contra instancias reales de CockroachDB mediante Docker Compose, verificando el comportamiento real de las consultas y transacciones frente al motor de persistencia utilizado en producción. En el nivel de extremo a extremo, la aplicación web se prueba con Playwright sobre tres motores de navegador (Chromium, Firefox y WebKit), y la aplicación móvil emplea pruebas instrumentadas con Espresso sobre un emulador Android API 34. Esta estrategia sigue las prácticas de pruebas automatizadas a gran escala descritas en [4] y responde a los retos específicos de prueba de sistemas basados en microservicios documentados en [5].

3.2. Pruebas sobre academico-laboratorios-service

El microservicio bajo responsabilidad del autor cuenta con 254 pruebas, alcanzando una cobertura de líneas igual o superior al 70 %, medida mediante el plugin `jacoco-maven-plugin` configurado en el ciclo de vida de Maven con la regla `jacoco:check`. Entre estas pruebas se incluye una verificación de contexto (`contextLoads`) que levanta el contexto completo de Spring Boot contra una instancia real de CockroachDB, no contra una base de datos en memoria, para garantizar que la configuración de persistencia es válida en condiciones equivalentes a producción.

3.3. Pruebas instrumentadas de la funcionalidad móvil de escaneo QR

Sobre la funcionalidad de escaneo de código QR de la aplicación móvil, implementada con CameraX y ML Kit Barcode Scanning, se desarrolló la suite `QrFlowTest`, compuesta por tres pruebas instrumentadas: verificación de que un código QR válido muestra el detalle del laboratorio correspondiente, verificación de que un identificador UUID inválido produce un mensaje de error apropiado, y verificación de que un error de red se comunica al usuario a través de un mensaje específico del *gateway*. Estas pruebas se ejecutan sobre un emulador Pixel 8 con API 34, debido a una incompatibilidad conocida entre Espresso 3.6.1 y las API 35 o superiores.

3.4. Cobertura verificada en el resto de microservicios

Al momento de este informe, la cobertura de código medida con JaCoCo está confirmada y documentada para dos de los cinco microservicios del sistema: `academico-laboratorios-service` (igual o superior al 70 %, 254 pruebas) y `reservas-solicitudes-service` (superior al 80 % en líneas y superior al 48 % en ramas, 110 pruebas). La cobertura de `usuarios-service`, `auth-service` y `api-gateway` no se reporta en este informe por no contar con verificación directa al momento de su redacción, en línea con el criterio de no presentar como medida un dato que no ha sido confirmado.

4. DevOps: CI/CD con GitHub Actions

4.1. Estructura general del pipeline

El pipeline de integración continua del sistema SCLI [6,7] está definido en `.github/workflows/ci.yml` y se activa ante eventos de `push` y `pull_request`. Su diseño sigue el principio de automatizar las etapas de análisis estático, construcción, pruebas y empaquetado antes de que un cambio pueda considerarse listo para integrarse a la rama compartida, reduciendo el riesgo de introducir regresiones no detectadas. El pipeline está compuesto por ocho jobs, descritos en las siguientes subsecciones.

Este diseño de pipeline multietapa coincide con las prácticas identificadas como más efectivas en la literatura sobre CI/CD para microservicios [8–10].

4.2. Jobs de análisis estático y pruebas

El job `lint` ejecuta Checkstyle sobre los cinco microservicios backend y ESLint sobre la aplicación web, verificando el cumplimiento de un conjunto de reglas de estilo personalizado que evita los excesos del estilo Google por defecto, cubriendo comprobaciones como `AvoidStarImport`, `UnusedImports`, `RedundantImport`, `NeedBraces`, `EqualsHashCode` y un límite de longitud de línea de 120 caracteres. El job `test-backend` ejecuta las pruebas unitarias y de integración de cada microservicio Java. El job `crdb-tests`, orientado a pruebas contra el clúster CockroachDB de la Entrega 3, permanece pendiente de completarse al momento de este informe. El job `test-web` ejecuta la suite de Vitest de la aplicación React, y el job `test-mobile` ejecuta las pruebas instrumentadas de la aplicación Android, incluida la suite `QrFlowTest` descrita en la Sección 3.

4.3. Jobs de construcción y empaquetado

El job `build-mobile-apk` compila y publica el artefacto APK de la aplicación móvil mediante `actions/upload-artifact`. El job `build-images`, que depende de la finalización exitosa de `test-backend`, construye y publica en el registro de contenedores de GitHub (GHCR) una imagen Docker por cada uno de los seis componentes desplegables del sistema (`auth-service`, `usuarios-service`, `academico-laboratorios-service`, `reservas-solicitudes-service`, `api-gateway` y `web`), mediante una matriz de compilación paralela que etiqueta cada imagen con el hash del commit correspondiente.

4.4. Job de integración completa

El job `integration`, que depende de `build-images`, levanta el sistema completo mediante `docker compose up -d --build` y espera activamente a que cada contenedor (bases de datos CockroachDB de cada dominio y los cinco microservicios) reporte estado `healthy` antes de continuar, con un máximo de sesenta intentos espaciados cada cinco segundos. Una vez que todos los servicios están saludables, el job verifica que el `endpoint /actuator/health` del `api-gateway` responda correctamente, confirmando que el sistema es capaz de arrancar y comunicarse de extremo a extremo bajo las mismas condiciones que tendría en un entorno de despliegue real.

4.5. Limitación identificada

Al momento de este informe no se cuenta con evidencia documentada de la demostración explícita que exige la guía: un *push* con un fallo intencional, seguido de la observación del pipeline en estado fallido, su corrección y la verificación del pipeline en estado exitoso. Esta demostración es un requisito pendiente que debería ejecutarse y documentarse con capturas antes de la defensa oral, dado que constituye condición necesaria según la propia guía de la práctica.

5. Observabilidad distribuida

5.1. Instrumentación implementada

El microservicio `reservas-solicitudes-service` expone, mediante Spring Boot Actuator, los puntos de salud y métricas requeridos por la plataforma de observabilidad. Micrometer publica tanto métricas HTTP estándar como eventos propios del dominio de negocio (solicitudes creadas, transiciones de estado, aprobaciones, rechazos, cancelaciones y propuestas), registrados únicamente después de confirmar la transacción correspondiente. Esta decisión de diseño es relevante: un *rollback* no debe contabilizarse como un evento de negocio exitoso, y un reintento idempotente no debe duplicar el conteo de una operación ya registrada.

Prometheus mantiene una configuración de *scrape* sobre el módulo de reservas, y el tablero de Grafana correspondiente reúne latencia HTTP, percentiles p50, p95 y p99, tasa de errores internos y contadores de operaciones de negocio. La instrumentación se complementa con cAdvisor, que aporta métricas de consumo de CPU, memoria y red a nivel de contenedor, siguiendo el enfoque de trazabilidad distribuida documentado en estudios industriales sobre observabilidad de microservicios [11].

5.2. Limitación identificada: campaña de carga no ejecutada

Al momento de este informe, los archivos de configuración y el cableado del servicio de observabilidad constituyen una base reproducible, pero **la campaña de pruebas de carga con Locust, necesaria para producir valores reales de percentiles de latencia y tasa de errores bajo tráfico simulado, no se ha ejecutado todavía**. El archivo de resultados previsto para esta campaña (`experimentos/resultados/iso25010.csv`) contiene la estructura completa de veinte repeticiones planificadas entre los escenarios de eficiencia (cincuenta usuarios, cinco minutos) y fiabilidad (cincuenta usuarios, una hora), pero todas las columnas de resultados (`total_requests`, `p95_ms`, `p99_ms`, `failure_rate_percent`) permanecen vacías.

Esta limitación se reporta de forma explícita, en lugar de completarse con valores estimados, porque cualquier cifra de latencia o tasa de error presentada sin haberse medido directamente constituiría una evaluación no honesta del sistema, contrario al criterio exigido por la guía de la práctica. La ejecución de esta campaña experimental queda identificada como el trabajo pendiente de mayor prioridad para completar tanto esta sección como la evaluación de las características de Fiabilidad y Eficiencia de desempeño de la Sección 6.

6. Evaluación de calidad ISO/IEC 25010:2023

7. Reflexión ética

8. Conclusiones

Referencias

- [1] S. Newman, *Building Microservices*. Sebastopol, CA, USA: O'Reilly Media, 2nd ed., 2021.
- [2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [3] C. Richardson, *Microservices Patterns*. Shelter Island, NY, USA: Manning Publications, 2018.
- [4] T. Winters, T. Manshreck, and H. Wright, *Software Engineering at Google*. Sebastopol, CA, USA: O'Reilly Media, 2020.

- [5] “Systematic mapping study of test generation for microservices: Approaches, challenges, and impact on system quality,” *Electronics*, vol. 14, no. 7, p. 1397, 2025.
- [6] J. Humble and D. Farley, *Continuous Delivery*. Boston, MA, USA: Addison-Wesley, 2010.
- [7] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook*. Portland, OR, USA: IT Revolution Press, 2nd ed., 2021.
- [8] M. Shahin, M. A. Babar, and L. Zhu, “Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices,” *IEEE Access*, vol. 5, pp. 3909–3943, 2017.
- [9] M. R. Pratama and D. S. Kusumo, “Implementation of continuous integration and continuous delivery (CI/CD) on automatic performance testing,” in *Proc. 9th Int. Conf. Inf. Commun. Technol. (ICoICT)*, pp. 230–235, 2021.
- [10] N. Thallapally, “Implementing continuous integration and continuous deployment (CI/CD) pipelines,” *International Journal of Science and Research Archive*, vol. 3, no. 2, pp. 248–253, 2021.
- [11] B. Li, X. Peng, Q. Xiang, H. Wang, T. Xie, J. Sun, and X. Liu, “Enjoy your observability: An industrial survey of microservice tracing and analysis,” *Empirical Software Engineering*, vol. 27, no. 1, p. 25, 2022.